

Rapport de projet TAL

UM4IN813 - Recherche d'Information et Traitement Automatique du Langage
Master MIND

CHEN Yuyu – 21500386
MNEMOI Lina – 21201897

Année universitaire 2025-2026

Table des matières

1 Introduction **4**

2 Présentation des données **4**
2.1 Commentaires de films 4
2.1.1 Analyse des longueurs des textes 4
2.1.2 Visualisation t-SNE 4
2.2 Présidents 6
2.2.1 Analyse des longueurs des textes 6
2.2.2 Visualisation t-SNE 7

3 Choix des modèles **8**

4 Preprocessing général **8**

5 Modèles linéaires : Régression et SVM **8**
5.1 Preprocessing 8
5.2 Pipeline : Vectorisation et classification 9
5.3 Entraînement et Résultats 9
5.4 Conclusion intermédiaire sur la méthode 10

6 Réseaux de neurones récurrents (Bi-LSTM) **10**
6.1 Entraînement et Résultats 10
6.2 Conclusion intermédiaire sur le modèle 11

7 Modèles transformers (DistilBERT et CamemBERT) **11**
7.1 Tokenisation 11
7.2 Modèle et fine-tuning 13
7.3 Résultats 13
7.4 Conclusion intermédiaire sur le modèle 14

8 Comparaison des performances **14**
8.1 Tâche 1 : Classification des films 14
8.2 Tâche 2 : Classification de style des présidents 15

9 Conclusion finale **15**

A Annexe **16**
A.1 Organisation du code 16
A.2 Principaux fichiers notebooks 16
A.3 Description des fichiers utils.py et preprocessing.py 17

Abstract

Ce rapport présente une étude comparative de modèles d'apprentissage appliqués à deux tâches distinctes de classification. La première tâche porte sur l'analyse d'un corpus de 27 000 critiques de films classées de façon binaire : sentiment positif ou négatif. La seconde tâche de classification consiste à construire un modèle capable de reconnaître le style linguistique de Chirac et Mitterrand afin d'estimer la probabilité que des phrases aient été dites par l'un ou par l'autre. Pour chaque tâche, nous avons évalué l'influence de plusieurs stratégies de prétraitement, de vectorisation et d'architectures de modèles, allant des approches linéaires aux réseaux de neurones profonds. Pour les deux tâches, le meilleure score en test est obtenu à l'aide des Transformers : le score F1 obtenu est de 0.92 pour la tâche de classification des critiques de films avec le modèle DistilBERT, et de 0.71 pour la reconnaissance de style oratoire, avec Camembert-large. Le couplage de la vectorisation TF-IDF et du SVM linéaire arrive juste derrière pour les films, avec un score F1 de 90.224, et 0.653 pour la seconde tâche.

1 Introduction

Ce projet de traitement automatique des langues (TAL) repose sur deux tâches de classification binaire. La première consiste à classifier des commentaires de films en sentiments positifs ou négatifs. La seconde vise à établir la probabilité que chaque phrase d'un ensemble de test a été dite par Chirac ou Mitterrand. L'entraînement est effectué sur un ensemble de phrases issues du débat du second tour de l'élection présidentielle de 1997. Dans ce rapport, nous détaillerons dans un premier temps les données utilisées ainsi que leur distribution. Nous présenterons ensuite les choix architecturaux de nos modèles, puis décrirons successivement les méthodes et modèles mis en œuvre. Enfin, nous procéderons à une comparaison des modèles retenus et de leurs performances.

2 Présentation des données

2.1 Commentaires de films

Notre dataset est composé de 27 000 revues de films, 2 000 fournies en TME 1 et 25 000 fournies en TME 4. La moitié de ces revues ont un sentiment général négatif, et l'autre un sentiment général positif.

2.1.1 Analyse des longueurs des textes

L'analyse des longueurs des commentaires révèle des caractéristiques intéressantes :

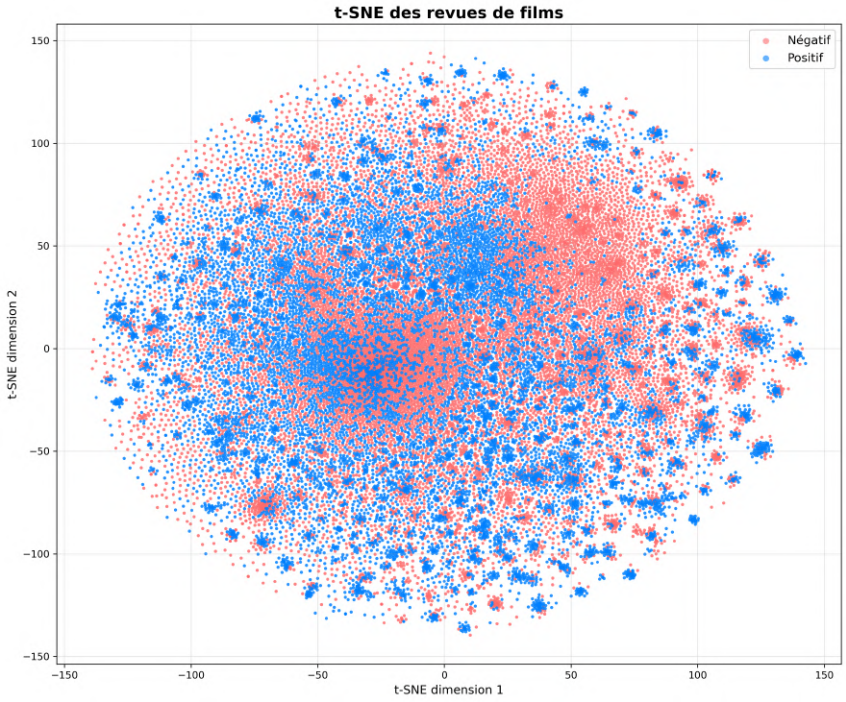
Table 1: Statistiques descriptives des longueurs des commentaires (en nombre de mots)

Classe	**Moyenne**	**Médiane**	**Écart-type**	**Min**	**Max**
Négatif	228.1	173.5	163.2	4	1 090
Positif	229.0	170.0	174.4	10	2 278
Ensemble	228.5	172.0	168.8	4	2 278

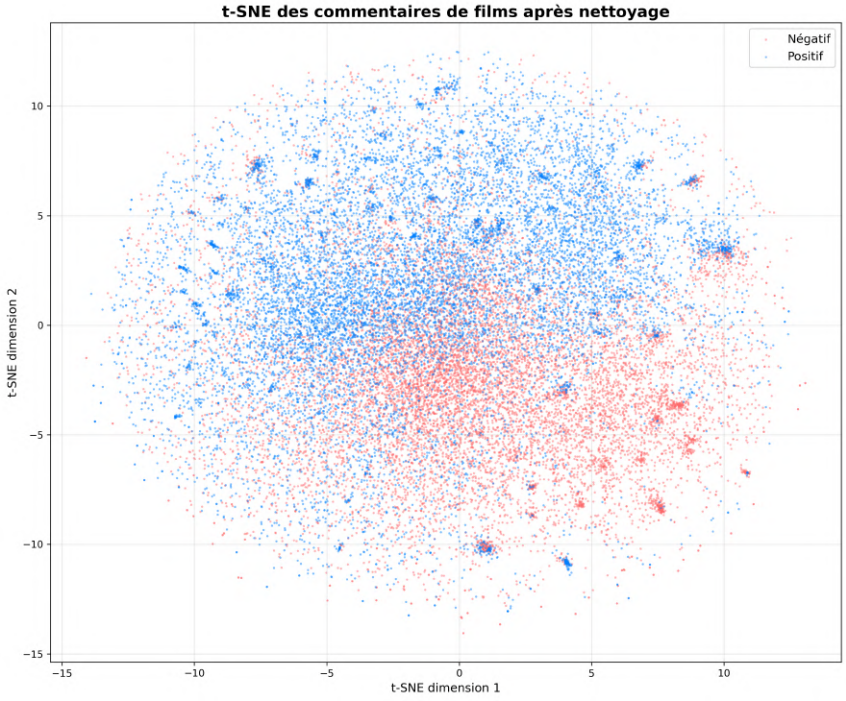
On observe que les longueurs moyennes sont très proches entre les deux classes (environ 228 mots). La médiane est légèrement inférieure à la moyenne (172 mots contre 228), ce qui indique une distribution asymétrique vers la droite : quelques critiques très longues tirent la moyenne vers le haut. Les critiques positives présentent une variabilité légèrement plus importante (écart-type de 174 contre 163 pour les négatives) et peuvent atteindre des longueurs beaucoup plus extrêmes (jusqu'à 2 278 mots contre 1 090 pour les négatives).

2.1.2 Visualisation t-SNE

Pour visualiser les données, on utilise la méthode t-SNE avec TF-IDF avant et après nettoyage (retrait des balises, lowerecasing, retrait de la ponctuation et lemmatisation).



(a) Avant preprocessing



(b) Après preprocessing

Figure 1: Visualisation t-SNE des commentaires de films

Nous observons sur la Figure 1a que les données présentent un recouvrement important entre les deux classes avant nettoyage. En effet, les points rouges et les points bleus sont très entremêlés et n'ont pas de frontière de séparation nette. Cependant, après nettoyage (Figure 1b), on peut observer une frontière de séparation allant de (-15, -5) à (15, 5). Même si elle ne sépare pas parfaitement les classes en deux en dimension 2, cela suggère que des modèles linéaires sur de plus hautes dimensions pourrait bien séparer les données. Sinon, des modèles plus riches tels que les RNN ou Transformers pourraient être capables de séparer les données.

2.2 Présidents

La base de données de la deuxième tâche est composée de 57413 phrases, dont 49890 prononcées par Chirac et 7523 énoncées par Mitterrand. La classe majoritaire est 6.6 fois plus représentée que la classe minoritaire, ce qui constitue un déséquilibre important. Ainsi, nous avons procédé à une augmentation des données en récupérant 400 discours, allocutions et messages de Mitterrand prononcés entre 1997 et 1993 sur le site `www.vie-publique.fr`. Il convient de noter que cette méthode ne garantit par l'absence de data leakage lors des prédictions sur les données de test. Cependant, cette préoccupation nous a semblé négligeable. En effet, il s'agit de personnalités historiques décédées. Le corpus des discours de Chirac et Mitterrand est définitivement figé et ne sera jamais actualisé. Ainsi, tout modèle entraîné sur ces données est nécessairement évalué sur le même univers fini, ce qui rend la séparation stricte entre sources d'entraînement et de test moins critique que dans un contexte évolutif.

2.2.1 Analyse des longueurs des textes

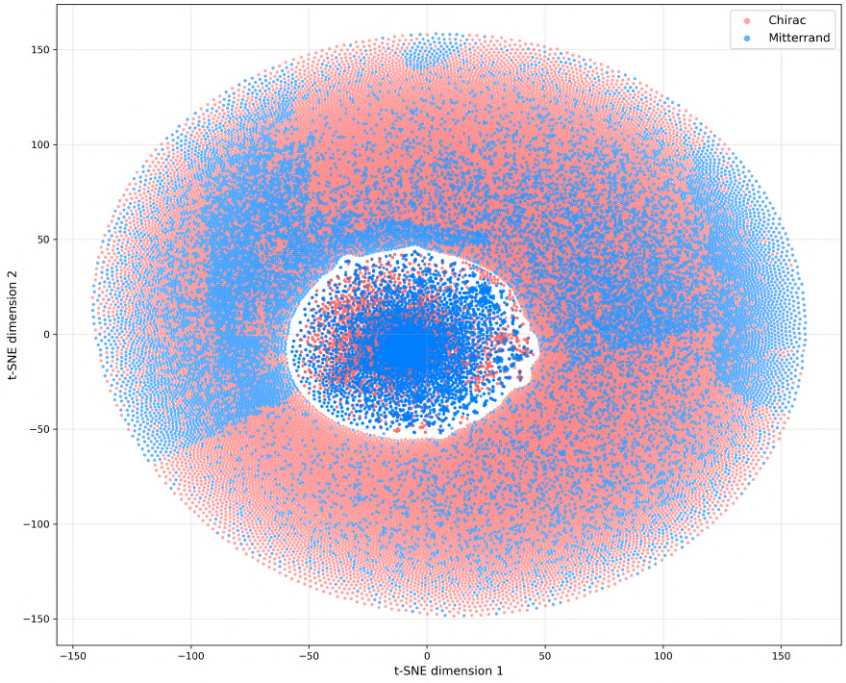
L'analyse des longueurs des interventions révèle une grande similarité dans la structure syntaxique des deux présidents :
Statistiques des longueurs par classe

Table 2: Statistiques descriptives des longueurs des textes (en nombre de mots)

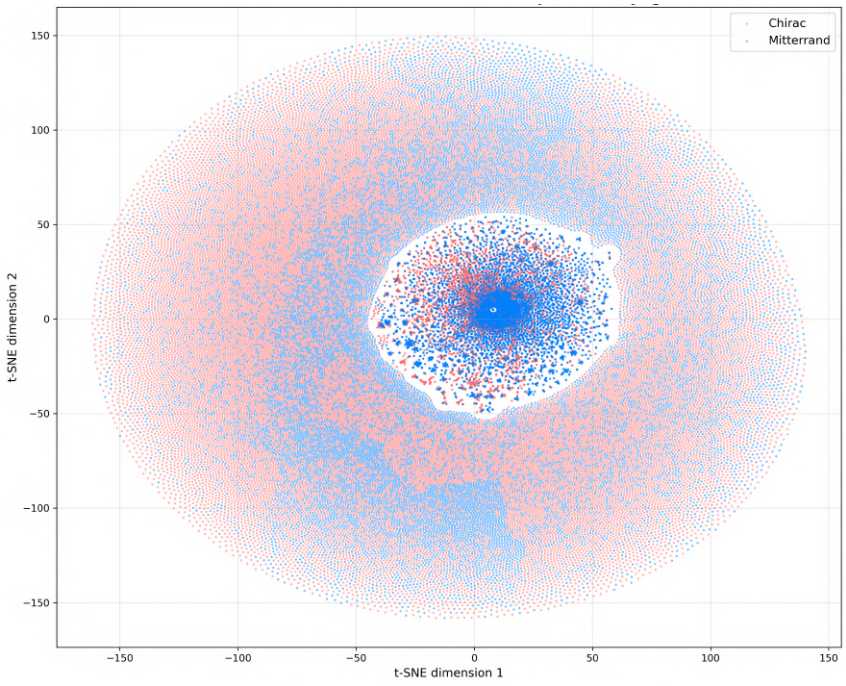
Classe	**Moyenne**	**Médiane**	**Écart-type**	**Min**	**Max**
Chirac	20.7	18.0	13.2	1	386
Mitterrand	22.8	18.0	17.4	1	297
Ensemble	21.6	18.0	15.5	**1**	**386**

On observe que les deux classes partagent une médiane identique (18 mots), ce qui suggère un rythme de phrase de base très proche entre les deux orateurs. La moyenne est légèrement supérieure à la médiane (21.6 contre 18.0), confirmant une distribution asymétrique vers la droite : la majorité des phrases sont concises, mais quelques interventions très longues tirent la moyenne vers le haut.
Bien que Mitterrand présente une moyenne légèrement plus élevée (22.8 mots), c'est chez Chirac que l'on trouve la valeur extrême la plus importante avec une phrase de 386 mots. L'écart-type plus élevé chez Mitterrand (17.4 contre 13.2) indique néanmoins une plus grande variabilité dans la longueur de ses interventions.

2.2.2 Visualisation t-SNE



(a) Avant preprocessing



(b) Après preprocessing

Figure 2: Visualisation t-SNE des phrases présidentiels

Sur la Figure 2a (corpus brut), on observe une structure circulaire très dense où les points rouges (Chirac) et bleus (Mitterrand) sont totalement mélangés. Contrairement à ce que nous avions noté pour les films, il n'existe ici aucune zone de séparation naturelle entre les deux locuteurs.
La Figure 2b (après nettoyage) montre que l'influence du prétraitement sur la répartition spatiale est quasi nulle. La superposition des classes reste importante au centre du graphique, même si des frontières de décision semblent se dessiner dans la partie qui l'entoure.
Ce mélange persistant souligne la difficulté de la tâche : les styles de Chirac et Mitterrand sont trop

proches pour être distingués par de simples clusters lexicaux en deux dimensions. Cela confirme la nécessité d'utiliser des modèles de haute dimension (SVM) et des architectures profondes (Transformers).

3 Choix des modèles

Face à nos problèmes de classifications, nous avons choisi trois types d'architectures de complexité croissante, que nous avons couplé à différentes méthodes de nettoyage et d'embedding :

- Modèles linéaires, en particulier SVM et TF-IDF : Ils constituent une baseline naturelle pour nos tâches. Leur simplicité les rend rapides, interprétables et relativement robustes au sur-apprentissage.
- Réseaux récurrents (LSTM et Bi-LSTM) : Bien qu'ils nécessitent un grand nombre de données pour être efficaces, nous avons décidé de les tester du fait de leur capacité à capturer l'ordre séquentiel des mots.
- Modèles Transformers (CamemBERT et DistilBERT) : Ils sont capables de capturer des relations contextuelles complexes et de comprendre le style d'un auteur. Ils se présentent étant comme parfaits pour nos tâches.

4 Preprocessing général

La performance des modèles de TAL dépend étroitement de la qualité du nettoyage des données. Comme chaque architecture possède des sensibilités différentes au bruit textuel, nous avons développé quatre niveaux de prétraitement :

1. Nettoyage minimal : Fonction se limitant à la suppression des espaces superflus. Elle sert de point de comparaison pour évaluer l'apport des différentes fonctions de nettoyage selon le modèle.
2. Nettoyage simple : Fonction appliquant un nettoyage minimal préservant ainsi au maximum l'information textuelle. Elle convertit le texte en minuscules, supprime les balises HTML, et conserve l'ensemble des lettres, apostrophes et tirets. Enfin, les espaces multiples sont normalisés.
3. Compromis entre nettoyage simple et agressif. A ce niveau, nous retirons les chiffres et la ponctuation.
4. Nettoyage agressif : Fonction destinée à réduire la dimension du vocabulaire et à ne conserver que les termes les plus informatifs. Après mise en minuscule et suppression des balises, elle supprime la ponctuation, les chiffres et caractères spéciaux. Les mots de moins de trois caractères et les tirets sont ensuite éliminés.

Ces différents niveaux de prétraitement ont été testés de manière itérative afin de déterminer le degré de nettoyage optimal pour chaque modèle.

5 Modèles linéaires : Régression et SVM

Les modèles linéaires sont les plus simples modèles et les plus interprétables. Ils nous servent de baseline pour nos tâches.

5.1 Preprocessing

Pour la classification des commentaires de films, nous avons retenu un nettoyage simple (minuscules, suppression des balises HTML et de la ponctuation). Les prétraitements plus agressifs, comme la lemmatisation ou la suppression des stopwords, ont systématiquement dégradé les performances.
Pour la classification des discours présidentiels, nous avons opté pour un nettoyage plus agressif : minuscules, lemmatisation, retrait de toute ponctuation. En validation, la simple lemmatisation a fait passer le F1-score de 0.8125 à 0.8307. Cependant, le retrait des stopwords diminuait les performances, probablement parce que certains mots génériques sont employés de manière différentielle par les deux locuteurs et conservent ainsi un pouvoir discriminant.

5.2 Pipeline : Vectorisation et classification

Pour les deux tâches de classification, l'ensemble des hyperparamètres a été choisi par `GridSearchCV`. cette méthode permet de tester conjointement différentes combinaisons d'hyperparamètres à l'aide d'une validation croisée, afin de sélectionner la configuration qui offre les meilleures performances.
Ainsi, pour la première tâche (films), nous avons retenu la configuration suivante : TF-IDF avec ngram_range = (1,3), min_df = 2, max_df = 0.8, sublinear_tf = True, et un LinearSVC avec C = 1.0.
Pour la deuxième tâche (présidents), nous avons utilisé `CalibratedClassifierCV` avec une méthode de calibration sigmoïde pour obtenir des probabilités en sortie. L'estimateur est un LinearSVC paramétré avec class_weight='balanced' pour gérer le déséquilibre des classes et C = 0.5. Par ailleurs, les résultats en validation croisée étant très similaires pour les valeurs de max_df comprises entre 0.6 et 0.9, nous avons conservé la valeur 0.8, couramment utilisée par défaut.

5.3 Entraînement et Résultats

Le split pour training et validation que nous avons utilisé est 80/20. Le SVM surpasse la régression linéaire pour les deux tâches. Nous présentons donc uniquement les résultats du SVM.

Table 3: Résultats en validation du SVM sur les commentaires de films (N : classe négative, P : classe positive)

F1-score	Accuracy	Precision N	Recall N	Precision P	Recall P
91.10%	91.10%	91.85%	90.20%	90.37%	92.00%

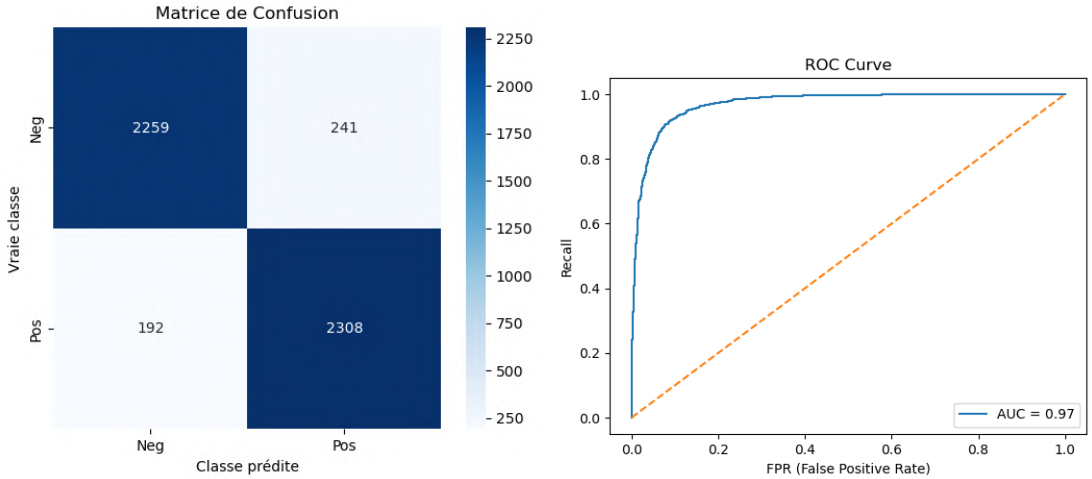


Figure 3: Matrice de confusion et courbe d'apprentissage pour la tâche 1

Interprétation : Le modèle ne fait presque pas d'erreurs et il est quasiment parfaitement capable de repérer le sentiment d'un commentaire de film. Cependant, des modèles capables de capturer le sens sémantique restent intéressants à explorer, même s'ils sont moins interprétables.
Note : Nous avons obtenu un score F1 de 99.984 en test, en incluant l'entièreté du dataset Stanford IMDB de HuggingFace. Nous ne prenons pas en compte ce résultat, par peur de data leakage.

Table 4: Résultats en validation du SVM calibré sur les phrases présidentielles

Données	**F1-score**	**Accuracy**	**Precision C**	**Recall C**	**Precision M**	**Recall M**
Baseline (données brutes)	77.78%	94.14%	94.32%	61.71%	60.95%	60.93%
Augm. data	81.25%	82.36%	83.64%	88.11%	80.11%	73.54%
Augm. data + lemmat	83.07%	84.03%	85.24%	89.03%	81.93%	76.34%

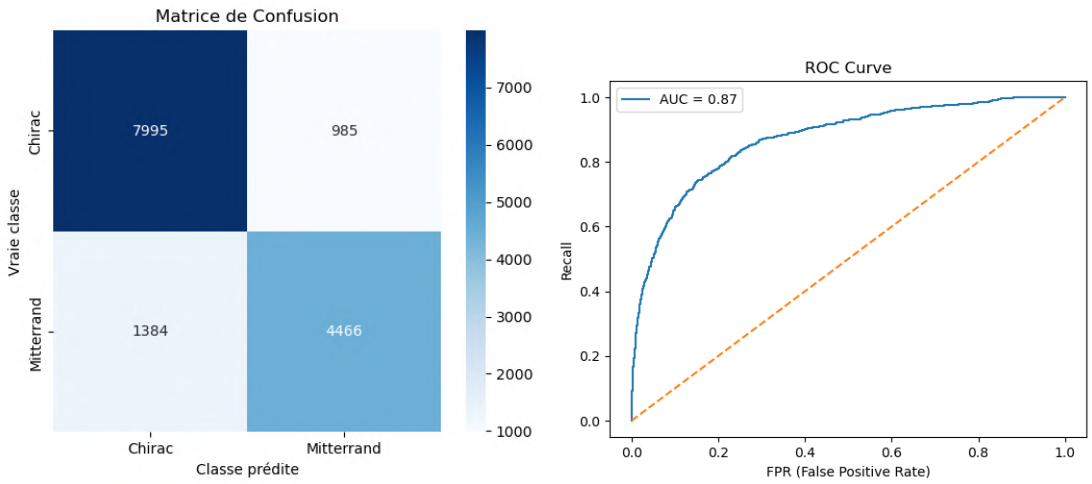


Figure 4: Matrice de confusion et courbe d'apprentissage pour la tâche 2

Interprétation : L'augmentation de données ainsi que la lemmatisation améliorent significativement le score F1. Les performances restent meilleures sur la classe majoritaire (Chirac, rappel de 89,03%) que sur la classe minoritaire (Mitterrand, rappel de 76,34%), illustrant l'incidence du déséquilibre persistant.

5.4 Conclusion intermédiaire sur la méthode

Le couple TF-IDF et SVM constitue une excellent baseline pour la classification des deux tâches. De plus, ils sont beaucoup plus explicables que d'autres modèles. Pour les films, il offre le meilleur compromis entre temps de calcul et performance (AUC de 0,99). Pour la seconde tâche, le résultat du SVM est déjà satisfaisant du fait de son interprétabilité. Cependant, les transformers et RNN semblent plus pertinents pour mieux capturer les nuances stylistiques et sémantiques propres à chaque président.

6 Réseaux de neurones récurrents (Bi-LSTM)

Nous avons utilisé une architecture Bi-LSTM pour les deux tâches de classification. Ce choix se justifie par la nécessité de capturer le contexte global de chaque phrase. Pour l'analyse de sentiment, le contexte bidirectionnel est crucial : une négation peut précéder ou suivre l'adjectif qu'elle modifie. Pour la reconnaissance du style, les motifs stylistiques et les tics de langage des orateurs ne sont pas purement linéaires. Ainsi, l'utilisation du Bi-LSTM permet au modèle d'accéder simultanément aux informations passées et futures pour chaque mot.
Pour optimiser ces modèles, nous avons mené plusieurs campagnes d'expérimentation faisant varier les méthodes de prétraitement ainsi que les configurations d'hyperparamètres.

6.1 Entraînement et Résultats

Le Bi-LSTM optimal a été entraîné avec les hyperparamètres suivants pour les deux tâches :

- Optimisation : AdamW avec un weight decay de 10^{-2} et un learning rate initial de 3×10^{-4}.
- Gestion du déséquilibre : Utilisation d'un WeightedRandomSampler pour la sélection des exemples et d'une fonction de perte Binary Cross-Entropy pondérée.
- Régularisation : Early stopping (patience de 4 époques sur le score F1) et gradient clipping (norme maximale de 1.0).
- Batch size : 64.

Les résultats obtenus sur la tâche de classification des sentiments selon le proprecessing appliqué sont présentés dans le tableau suivant :

Table 5: Résultats du Bi-LSTM sur les commentaires de films en validation

Prétraitement	Gestion des Stopwords	Accuracy (Train)	Accuracy (Val)
Minimial	Sans topwords	0.784	0.566
Simple	Custom stopwords	0.841	0.644
Simple	Sans stopwords	0.769	0.553
Entre simple et agressif	Custom stopwords	0.873	0.675

On observe que la meilleure performance (67,5% d'accuracy en validation) est obtenue avec le prétraitement modéré (retrait des chiffres, balises et de la ponctuation) et l'utilisation de stopwords personnalisés. On observe cependant un sur-apprentissage marqué : alors que l'accuracy sur le jeu d'entraînement dépasse les 87%, elle plafonne aux alentours de 67% en validation. Ce phénomène était attendu compte tenu du nombre relativement limité de données disponibles pour un modèle de cette complexité. Pour la

tâche de classification des discours présidentiels, nous avons entraîné un Bi-LSTM avec quatre variantes de prétraitement, décrites dans la section Preprocessing commun. Les résultats sont les suivants :

Table 6: Résultats du Bi-LSTM sur les discours présidentiels en validation

Prétraitement	Accuracy	F1-score	Precision	Rappel
Minimal	0.847	0.520	0.442	0.631
Simple	0.851	0.525	0.452	0.627
Entre simple et agressif	0.860	0.523	0.472	0.586
Nettoyage agressif	0.836	0.478	0.411	0.572

La meilleure accuracy (86,0%) est obtenue avec le prétraitement simple. Cependant, le F1-score reste faible (environ 0,52), ce qui s'explique par le fort déséquilibre des classes. Le modèle a tendance à privilégier la classe majoritaire et manque probablement de données pour mieux classer les phrases. Utiliser un dropout plus élevé ou augmenter la taille des données pour la classe minoritaire pourrait atténuer le problème. Compte tenu de la supériorité du SVM (F1-score de 0,83) et de sa plus grande interprétabilité, l'usage d'un modèle de type black box moins performant n'apparaît pas justifié pour cette tâche spécifique.

6.2 Conclusion intermédiaire sur le modèle

Les expériences menées avec les réseaux de neurones récurrents montrent que ces modèles, bien que prometteurs sur le plan théorique, peinent à surpasser les approches linéaires sur nos jeux de données. Le principal facteur limitant est la taille modeste des corpus d'entraînement. En effet, celui-ci conduit ici à un sur-apprentissage malgré l'utilisation de techniques de régularisation (Dropout, Weight Decay et Early Stopping). Ce constat nous condient à explorer les modèles Transformers, qui sont pré-entraînés avec une large quantité de données.

7 Modèles transformers (DistilBERT et CamemBERT)

Les modèles Transformers sont capables de modéliser simultanément toutes les relations entre les mots d'une phrase. Par conséquent, ils sont capables le style d'un auteur, et le sentiment général d'une phrase ou d'un texte. Ils se présentent donc comme de parfaits candidats pour nos tâches.
Les modèles BERT sont pré-entraînés sur différents langages. Pour les films, nous avons utilisé DistilBERT. Pour les présidents, nous avons utilisé CamemBERT-large, modèle pré-entraîné spécifiquement sur le français.

7.1 Tokenisation

Contrairement aux autres modèles, les Transformers ne nécessitent pas de prétraitement manuel. La tokenization est assurée par le tokenizer propre à chaque modèle.
Pour la tâche des commentaires de films (en anglais), nous avons utilisé le tokenizer de `distilbert-base-uncased`. La longueur maximale a été fixée à 512 tokens. Ce choix est basé sur l'analyse de la distribution des

longueurs (Figure 5), fortement asymétrique avec une queue s'étendant jusqu'à environ 1000 mots. Au-delà de 400 mots, les observations deviennent rares (moins de 1000 par tranche de 100 mots). Une limite à 256 aurait donc tronqué une part non négligeable des textes de la zone principale de distribution, tandis que 512 permet de conserver l'essentiel de l'information.

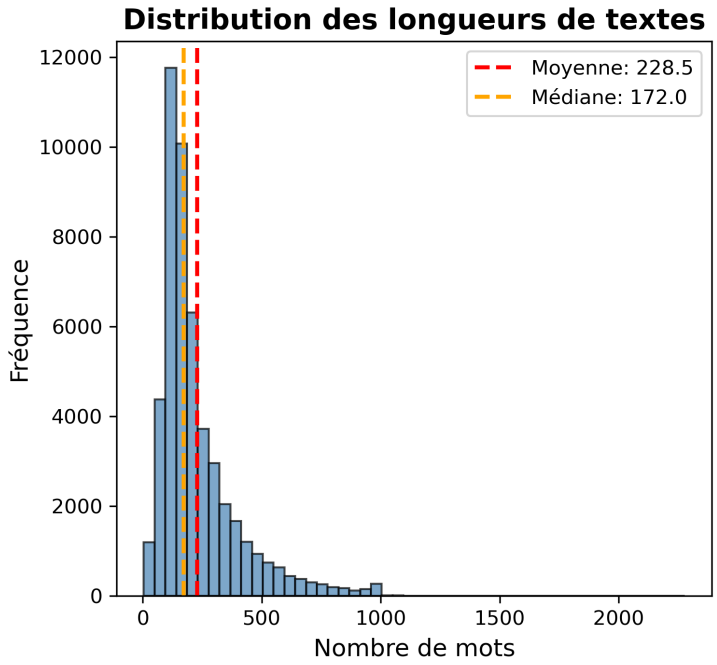


Figure 5: Distribution des longueurs des critiques de films

Pour la tâche des présidents, la moyenne des phrases est de 21,6 mots les plus courtes contenant un seul mot, et la plus longue 386 mots. Ainsi, nous avons choisi un maximum de 128 tokens par phrase, ce qui couvre la grande majorité des phrases du corpus (voir Figure 6). Les phrases dépassant cette limite ont été tronquées, tandis que les plus courtes ont été complétées par du padding. Nous avons utilisé le tokenizer de `camembert/camembert-large` pour le modèle final, après avoir testé `camembert-base`.

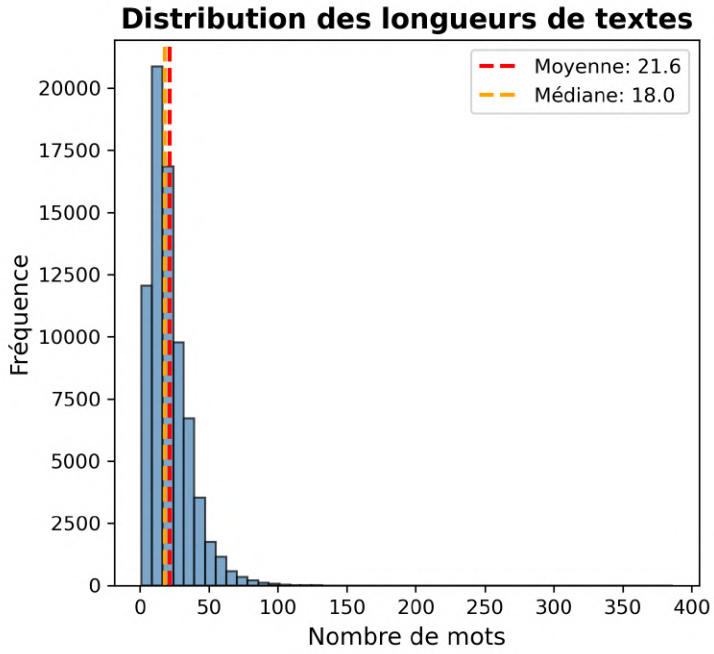


Figure 6: Distribution des longueurs des phrases des présidents

7.2 Modèle et fine-tuning

Pour les hyperparamètres, nous avons fait plusieurs tests (recherche manuelle). Nous avons utilisé les mêmes hyperparamètres pour les deux modèles :

Table 7: Hyperparamètres du fine-tuning pour DistilBERT base

Paramètre	**Valeur**
Modèle	distilbert-base-uncased
Taille de batch	16
Taux d'apprentissage	10^{-5}
Nombre d'époques	3
Longueur maximale	256 pour les films, 128 pour les présidents
Warmup steps	10% des steps totaux
Weight decay	0.01
Label smoothing	0.05
Patience early stopping	3 epoch

Néanmoins, gérer le déséquilibre des classe, nous avons appliqué une pondération sur la fonction de perte de Camembert. En effetn nous avons multiplié le poids de la classe minoritaire (Mitterrand) multiplié par 2.0 (valeur trouvée après plusieurs jeux) et nous avons défini un textttlabel_smoothing de 0.05 pour améliorer la généralisation.

7.3 Résultats

Le fine-tuning de DistilBERT sur les critiques de film a produit les résultats suivants en validation :

Epoch	**Train loss**	**Val loss**	**Accuracy**	**F1**	**Precision**	**Recall**
1	0.3165	0.2117	0.9192	0.9206	0.9046	0.9372
2	0.1664	0.2393	0.9264	0.9279	0.9091	0.9476
3	0.1195	0.2832	0.9250	0.9251	0.9242	0.9260

On observe que la loss de validation est minimale dès la première époque (0.2117), puis commence à augmenter progressivement, ce qui indique un début de sur-apprentissage malgré une amélioration de

l'accuracy globale à l'époque 2. Nous avons donc conservé les poids issus de la première époque pour garantir une meilleure généralisation.
Sur l'ensemble de test, DistilBERT affiche des performances solides avec un score F1 de 0.92. Bien que ces résultats soient excellents, ils restent très proches de ceux obtenus avec le modèle SVM linéaire. Cela confirme que pour cette tâche spécifique de classification de sentiment, l'apport d'une architecture Transformer, bien que performante, est limité par rapport à une approche TF-IDF/SVM beaucoup plus légère en termes de ressources computationnelles. Le fine-tuning de CamemBERT-large sur les phrases

présidentielles a produit les résultats suivants :

Epoch	Train loss	Val loss	Accuracy	F1	Precision	Recall
1	0.3449	0.3227	0.8953	0.661	0.574	0.781
2	0.2141	0.3748	0.9227	0.715	0.691	0.741
3	0.1457	0.4509	0.9256	0.728	0.699	0.759

On observe que la loss de validation commence à augmenter après la première epoch, signe d'un début de sur-apprentissage. Nous avons donc conservé ce modèle. Sur l'ensemble de test, nous avons obtenu un F1-score de 0.71, avec une AUC-ROC de 0.93. Cela signifie que le modèle parvient plutôt bien à distinguer les deux classes. CamemBERT-large performe mieux que le SVM pour cette tâche (+ 6 points en F1-score).

7.4 Conclusion intermédiaire sur le modèle

Ainsi, les performances de DistilBERT restent comparables à celles du SVM. La structure lexicale simple du sentiment (mots-clés positifs/négatifs) permet au TF-IDF de rester extrêmement compétitif. Cependant, pour la reconnaissance de style des présidents, les Transformers surpassent nettement toutes les autres approches.

8 Comparaison des performances

8.1 Tâche 1 : Classification des films

Le tableau suivant synthétise les performances et les coûts en ressources de nos différentes approches pour l'analyse de sentiment :

Table 8: Comparaison des modèles sur la classification des critiques de films

Modèle	Accuracy	F1-score	Observations	Temps train moyen
SVM linéaire (Test)	0.9026	0.9022	Excellent rapport perf/coût	3 min
Bi-LSTM (Val)	0.675	0.659	Sur-apprentissage marqué	20 min
DistilBERT (Test)	0.920	92.127	Très performant	18 min

Note : les résultats Bi-LSTM correspondent au jeu de validation (aucune soumission test effectuée sur la plateforme).

Le modèle DistilBERT obtient le meilleur score en test avec une accuracy atteignant les 92% et un F1 score les dépassant. Les RNN souffrent d'un surapprentissage important (écart de 25 points entre train et validation). Le modèle ne parvient pas à généraliser. En outre, le SVM produit un score quasiment égal à celui des Transformers. Ainsi, bien que très performant, le modèle DistilBERT présente un ratio performance/coût nettement moins favorable au SVM. En effet, si ses résultats sont similaires à ceux du SVM, son temps d'entraînement est plus de 6 fois supérieur (18 minutes contre 3 minutes). L'écart est encore plus flagrant pour l'inférence : la prédiction sur l'ensemble du jeu de test prend environ 180 secondes pour DistilBERT, contre seulement 15 secondes pour le SVM. Le modèle DistilBERT est 30 fois plus lent. Toutefois, compte tenu de sa complexité, son temps d'exécution demeure dans l'absolu relativement efficace.
En conclusion, pour cette tâche de classification binaire, la complexité des modèles de Deep Learning n'apporte pas de gain significatif par rapport à une approche simple, robuste et rapide (vectorisation et modèle linéaire).

8.2 Tâche 2 : Classification de style des présidents

Table 9: Comparaison des modèles sur la classification Chirac vs Mitterrand

Modèle	Accuracy	F1-score	Precision (M)	Rappel (M)	Temps train moyen
SVM linéaire calibré	0.880	0.68	0.59	0.81	6 min
Bi-LSTM	0.860	0.52	0.47	0.59	20 min
CamemBERT-large	0.926	0.73	0.70	0.76	75 min

CamemBERT-large surpasse les autres approches, avec un gain de 4,6 points d'accuracy et 5 points de F1-score par rapport au SVM. Le principal avantage des Transformers réside dans leur capacité à mieux traiter la classe minoritaire. La précision sur la classe Mitterrand passe de 0,59 (SVM) à 0,76 (CamemBERT), soit une amélioration de 17 points. Cependant, l'entraînement prend 75 minutes en moyenne contre 6 pour le SVM. En somme, si le SVM reste un excellent compromis entre temps de calcul et efficacité pour des analyses rapides, les Transformers constituent la solution de référence pour atteindre une précision maximale. Leur utilisation est justifiée dès lors que des ressources GPU sont disponibles.

9 Conclusion finale

En conclusion, l'efficacité des méthodes de prétraitement s'est avérée dépendante de la nature des tâches : un nettoyage minimal a suffi pour l'analyse de sentiment (films), tandis que la lemmatisation n'a apporté un gain significatif qu'au modèle SVM sur le corpus des discours présidentiels. Les modèles linéaires (surtout le SVM) confirment leur statut de références : ils sont simples, rapides à entraîner et robustes face au sur-apprentissage. À l'inverse, les réseaux récurrents (RNN), malgré leur complexité technique, n'ont pas réussi à surpasser ces approches classiques sur nos données, leur taille étant modeste. Enfin, les Transformers offrent les meilleures performances globales ; ils constituent le choix idéal si l'on dispose d'un GPU. Toutefois, le duo SVM/TF-IDF demeure une alternative très efficace et représente le meilleur compromis entre temps de calcul, interprétabilité et précision pour de l'apprentissage sous contraintes de ressources et de temps.

A Annexe

Cette annexe décrit l'architecture du code du projet, l'organisation des fichiers et les principales fonctions implémentées.

A.1 Organisation du code

Ci-dessous l'organisation du code complet. Seuls les dossiers Codes et Results sont disponnibles sur GitHub (/github.com/Chenyu1016/RITAL-PROJECT-SHARE), sans les graphiques et courbes.

```
RITAL-PROJECT-SHARE/
 Codes/
    Dialog/                      # Tâche 2
       methods/                 # Fonctions de prétraitement et utilitaires
          preprocessing.py
   |    mitterrand.txt          # Données d'augmentation
       main_pres.ipynb          # Entraînement & Évaluation

    Movies/                      # Tâche 1 : Critiques de films
        methods/                 # Fonctions de prétraitement et utilitaires
           utils.py
        Reports/                 # Graphiques, courbes et matrices de confusion
        main_movies.ipynb        # Entraînement & Évaluation

 Models/                          # Sauvegarde des meilleurs modèles
    Dialog/
    Movies/

 Ressources/                      # Données
    Dataset/
       corpus.tache1.train.utf8
       Movies/                  # Les 2000 critiques de films (TME 1)
    Projet_Test_Set/            # Données de test fournies
       corpus.tache1.test.utf8
       testSentiment.txt
    english_contractions.json
    movie_dataset.txt
    number_list.txt

 Results/                         # Fichiers de soumission
     submission-movie-[1-4].csv
     submission-pres-[1-5].csv
```

A.2 Principaux fichiers notebooks

Nous décrivons ici l'ensemble des notebooks déposés sur Moodle. L'exécution de ces scripts suppose le respect de l'arborescence de fichiers présentée précédemment pour assurer le bon chargement des dépendances et des données.

`cam.ipynb` : Ce fichier est utilisé pour le fine-tuning du modèle CamemBERT avec Google Colab. Il comprend :

- Importation des bibliothèques ;
- Définition de la classe `TransformerWrapper` pour encapsuler la logique du modèle ;
- Intégration du jeu de données étendu incluant le fichier `mitterrand.txt` qui contient 16 737 phrases supplémentaires ;
- Configuration du Trainer (hyperparamètres et stratégie d'évaluation) ;
- Phase d'entraînement et génération des prédictions.

`dist.ipynb` : Ce fichier est utilisé pour le fine-tuning du modèle DistilBERT avec Google Colab. Il comprend :

- Importation des bibliothèques ;
- Définition de la classe `TransformerWrapper` pour encapsuler la logique du modèle ;
- Chargement du corpus de 25 000 critiques de films donné en TME 4 ;

- Configuration du Trainer (hyperparamètres et stratégie d'évaluation) ;
- Phase d'entraînement et génération des prédictions.

`main_pres.ipynb` : Ce notebook est le notebook principal pour l'exploration et le test des modèles linéaires et RNN de la tâche 2. Il comprend :

- Chargement et courte analyse des données ;
- Mise en œuvre du pipeline tf-idf + svm avec GridSearchCV ;
- Exécution de campagnes d'expériences automatisées testant itérativement différentes fonctions de prétraitement et modèles (Régression logistique, SVM, RNN, Transformer), en utilisant des fonctions définies dans `preprocessing.py` que nous décrivons dans la section suivante.

`main_movies.ipynb` : Ce notebook est le notebook principal pour l'exploration et le test des modèles linéaires et RNN de la Tâche 1. Il comprend :

- Chargement et courte analyse des données ;
- Mise en œuvre du pipeline tf-idf + svm avec GridSearchCV ;
- Exécution de campagnes d'expériences automatisées testant itérativement différentes fonctions de prétraitement et modèles (Régression logistique, SVM, RNN, Transformer), en utilisant des fonctions définies dans `utils.py` que nous décrivons dans la section suivante.

A.3 Description des fichiers utils.py et preprocessing.py

`utils.py` : L'ensemble des fonctions définies dans ce fichier constitue le cœur du pipeline expérimental construit pour la tâche 1 (Films). Il comprend :

- Fonctions de nettoyage : Définition de quatre niveaux de prétraitement (de clean_raw à clean_keep_negation ;
- Gestionnaire d'expériences : Implémentation de la structure `ExperimentResult` pour le suivi systématique des métriques (Accuracy, F1-macro, Log-Loss) et des chemins de sauvegarde ;
- Fonctions de construction dynamique de pipelines *Scikit-Learn* (Tfidf + SVM/LogReg), d'architectures Keras (BiLSTM) et de Transformers (DistilBERT) ;
- Outils de validation croisée stratifiée (`StratifiedKFold`), génération automatique de matrices de confusion et algorithme de sélection du meilleur modèle global (`copy_best_models`) ;
- Fonction predict_with_best_global permettant de charger le modèle optimal et de générer le fichier de soumission CSV final.

`preprocessing.py` : L'ensemble des fonctions définies dans ce fichier constitue le cœur du pipeline expérimental construit pour la tâche 2 (Présidents). Il regroupe les utilitaires de traitement et les architectures de modèles. Il comprend :

- Fonctions de nettoyage : Définition de cinq niveau de prétraitement allant de clean_1 à clean_4 et une fonction preprocess_pres qui procède à la lemmatisation via SpaCy ;
- Wrappers de modèles :
  - `W2VLogRegWrapper` : Implémentation d'un modèle Word2Vec pondéré par TF-IDF avec entraînement itératif et early stopping (performances pas convaincantes donc non inclus dans le rapport) ;
  - `RNNWrapper` : Architecture Bi-LSTM sous PyTorch incluant un WeightedRandomSampler pour gérer le déséquilibre des classes ;
  - `TransformerWrapper` : Intégration de CamemBERT via la bibliothèque HuggingFace, avec une personnalisation de la fonction de perte (`WeightedTrainer`) ;
- Fonctions de calcul de métriques (Accuracy, F1, Log-Loss), validation croisée stratifiée et génération de matrices de confusion formatées ;
- Système de sauvegarde et de chargement gérant les fichiers `.joblib` pour Scikit-Learn et les répertoires de poids pour les modèles de Deep Learning.